

Colin Hogan

PHYS 537

Problem 1 – Two-site LCAO with finite overlap

1. The construction of H and S , as well as the generalized eigen-values/-vectors, are calculated by the code and printed to the console.
2. There is then code written to make sure the eigenvectors are each normalized such that $\mathbf{c}^T \mathbf{c} = 1$ and that the matrix of eigenvectors is normalized relative to the inner product $\mathbf{C}^T \mathbf{S} \mathbf{C} = \mathbf{1}$.
3. E_+ and E_- are also calculated by the code and printed to the console. The symmetric bonding and antisymmetric antibonding states are calculated using the equations from the textbook and printed to the console. Notice that these energies are the same as the eigenvalues calculated before.
4. The calculation is repeated with $\mathbf{S} = \mathbf{1}$, and the results are printed to the console.
5. Then there's code to plot the energies on the same figure as a function of s , where s ranges from 0.00 to 0.40. The point of interest (where $s = 0.18$) is also marked on this plot. The plot is then converted into a 600-DPI PNG.
6. A typical eigenvalue problem occurs when the eigenvectors form an orthogonal basis for symmetric square matrix on the left side of the equality. In an orthogonal basis, the inner product is just the usual dot product (or its complex equivalent), but in a nonorthogonal basis, the inner product is not so simple and involves some operator. It's like the calculation of $\langle p \rangle$, the average momentum in quantum mechanics which uses the $\hat{p}$ operator between the eigenstate and its conjugate to calculate the inner product. $\mathbf{S}$ effectively provides the operator that allows the eigenvectors to be orthogonal in $\mathbf{H}$ in this problem. The use of $\mathbf{S} = \mathbf{I}$ is a model approximation because it assumes there is no overlap, which is the case when the two atoms are very far apart and the solutions to the eigen-problem are those of an isolated atom.

Problem 2 – Exact singlet of the Hubbard dimer

1. The Hamiltonian is constructed and diagonalized for every assigned U/t . Each Hamiltonian and its diagonalization are printed to the console.
2. E_{GS}/t , the ionic probability $|c_I|^2$, and the exact singlet-triplet gap J/t are then calculated by the code and printed to the console. It is noted that the ionic probability is given by $c_I = \sin \theta$, where $\theta = \frac{\tan^{-1} \sqrt{U^2 + 16t^2}}{4t}$. The ground state energies were the lowest eigenvalue of each Hamiltonian, which also can be calculated by the equation given in the problem.

3. The numerical ground state energies (lowest eigenvalue of each Hamiltonian) were verified against the evaluation of the formula given for the respective U by the code. The user can see that each evaluation of the E_{GS} formula is equal to the lowest eigenvalue of the respective Hamiltonian. Both values are printed to the console.
4. For $U > 0$, the exact J was calculated with the given formula for J for each U/t . So too were the large- U superexchange estimates. These values were compared using percent error. The user can see J , superexchange, and percent error for each U/t . As U/t increases, J and superexchange are less different, which is expected from a large- U estimate.
5. Over the range $0.25 \leq U/t \leq 20$, J/t and the superexchange estimate divided by t were plotted vs U/t on the same plot. At very low U/t , they are quite disparate, but quickly approach one another by about $U/t = 5$. Both approach zero at very large U/t .
6. U is the energy added for each doubly occupied site. It goes to zero when the site is singly occupied or unoccupied. But as more doubly occupied sites are present, each electron experiences more on-site repulsion, so U must increase. Larger U/t represents larger doubly occupancy. One can see from the ionic probability plot that as U/t becomes very large, the ionic probability becomes very small and tends toward zero. The state becomes more covalent when U/t is large.

Problem 3 – Slater-Koster matrix for an arbitrary bond

1. The bond length is calculated in the code from the components of the bond vector. Dividing the components of the bond vector by this value gives the components of the unit bond vector. These components are also called the direction cosines. These and the bond length are reported in the console. The code also verifies that the sum of the squares of the direction cosines is 1, the expected length of a unit vector. This value is reported in the console as well, though it isn't 1. It's 0.9999999, which I suppose is due to some floating-point magic.
2. The Harrison coefficients are then calculated and printed to the console as "Potentials". They are in electron volts (eV).
3. The complete 4-by-4 intersite block is also constructed first as a numpy array and then printed to the console as a pandas data frame with row and column labels. Both sets of labels are just the local basis $\{s, p_x, p_y, p_z\}$ since each site has the same basis.
4. The bond is reversed by making the bond vector's components negative. This new intersite block is constructed in the exact same way as the original one. This data frame is also printed to the console. Next, the intersite block for the original bond vector is transposed and printed to the console. The code checks that each entry in the reverse bond block is the same as in the not-reversed bond transposed block and reports in the console if they are equal to numerical precision.

5. The code also looks at the not-transposed not-reversed intersite block and the reverse-bond intersite block for entries in the same position that are negatives of each other (except for zero-entries, since -0 is obviously the same as 0 and that would be confusing). Then it reports to the console which entries become negative when the bond is reversed. This report is given in coordinate notation with the basis labels. For example, if the entry in the second row and first column (in human language, not computer scientist language) is made negative (which it is), then it would be reported that (p_x , s) changes under bond reversal. Actually, the only elements that change sign under bond reversal are s-p elements. The s-s element and the p-p block are unchanged. The intersite block (without bond reversal) is plotted as a heat map that's saved as a 600-DPI PNG.
6. Each p-orbital has two opposite phase lobes. When two orbitals overlap, they'll have a sign that represents whether the overlap is constructive or deconstructive. Since the s-orbital is spherically symmetric, reversing the bond doesn't do anything to the sign. It's even under bond reversal. But since the p-orbitals have opposite phase lobes, they aren't spherically symmetric. They're symmetric about a 180° rotation and a sign change, meaning the p-orbitals are odd under bond reversal. So reversing the bond would effectively just change the sign of the orbital. Since the s-orbital is even under bond reversal, the s-s overlap is even, so it doesn't change under bond reversal. Since the s-orbital is even and each p-orbital is odd under bond reversal, the s-p overlaps are overall odd, so they change sign under bond reversal. Since the p-orbitals are each odd under bond reversal, the p-p overlaps are overall even (negative times a negative is a positive). The orbitals' parities explain why all the s-p elements change sign, but the p-p block (including the zero-entries) don't change sign.

Problem 4 – O₂ tight binding, cohesion, and magnetism

1. The complete 8-by-8 sp Hamiltonian and its diagonalization are constructed for several O-O separations. The separations are symmetric shifts from $d_0 = 1.208$ angstrom. The matrices are printed to the console. For this problem, I would recommend looking at only one iteration of this because it's a bit long for reading in the console. The function that constructs and diagonalizes the Hamiltonian also checks that it's Hermitian by transposing the Hamiltonian and making sure that each element is equal to the entry in the same position in the transposed Hamiltonian. Another function classifies the eigenvectors as σ , σ^* , π , or π^* . The pi states come from checking whether the eigenstate's corresponding eigenvalue comes from adding or subtracting $V_{pp\pi}$ from ϵ_p^0 . The sigma states come from the equations

$$E = \frac{\epsilon_s^O + V_{ss\sigma} + \epsilon_p^O - V_{pp\sigma}}{2} \pm \sqrt{\left(\frac{\epsilon_s^O + V_{ss\sigma} - \epsilon_p^O - V_{pp\sigma}}{2}\right)^2 + V_{sp\sigma}^2}$$

$$E = \frac{\epsilon_s^O - V_{ss\sigma} + \epsilon_p^O + V_{pp\sigma}}{2} \pm \sqrt{\left(\frac{\epsilon_s^O - V_{ss\sigma} - \epsilon_p^O + V_{pp\sigma}}{2}\right)^2 + V_{sp\sigma}^2}$$

which I got from page 43 of *Elementary Electronic Structure* by Walter A. Harrison (1999). The pi eigenvalues are also degenerate – two pi eigenstates for each pi eigenvalue.

2. The 12 valence electrons (6 from each O) are placed into molecular orbitals using the Pauli exclusion principle and Hund's rule. I made an array than contains lists of different sizes to account for the number of electrons that can go in each energy level. Sigma lists are each length 1, and pi lists are each length 2. Their positions in the list come from the least to greatest order of the eigenvalues and their correspondence to sigma or pi eigenstates. The lowest sigma list fills before the next one and so on, and a pi list fills by counting each position in the list up to one before counting each up to two and then moving on to the next list. The function that fills this out also uses it to construct a string that gives the molecular orbital configuration, determine the number of paired and unpaired electrons, and calculate the bond order. The filling list, molecular orbital configuration, bond order, classification of states, and *spoilers* E_{band} are printed to the console.
3. If you didn't skip past the spoiler, you already know that the last function mentioned also calculates E_{band} . It does this by summing up the eigenvalues weighted by the number of electrons at each energy level, which it gets from the molecular orbital configuration. Each E_{band} is stored in an array and use in combination with the array of bond distances constructed earlier to estimate a symmetric derivative of E_{band} with respect to bond distance. Calculating the derivative of E_{tot} with respect to bond distance and setting it equal to zero is the usual method of calculating the coefficient C in V_{rep} . This sets the derivative of E_{band} equal to the derivative of V_{rep} . The estimated symmetric derivative of E_{band} is then multiplied by $\frac{d^5}{4}$, where these values are calculated at $d = d_0 = 1.208$ angstrom. This gives the value of C. Then E_{tot} and ΔE can be formed from E_{band} and V_{rep} using the formulae given in this problem's prompt. The value of C is printed to the console. E_{tot} and ΔE don't have general forms since E_{tot} doesn't have a general form except for the one given in the prompt, but I don't think that's what this task wanted. It's noted in the console what the calculation of each is, and these forms are used to make plots.
4. A linspace was made to represent various finely spaced bond distances. These bond distances were used to fill up an array representing E_{coh} . These values were calculated by

subtracting the bond-distance-dependent E_{tot} from the constant E_{sep} . A plot was made of E_{coh} vs bond distance, and a maximum value is seen between the endpoints. The bond distance for which E_{coh} is a maximum is the equilibrium bond distance, and the maximum E_{coh} is the model cohesive energy. The equilibrium bond distance and the model cohesive energy are printed in the console.

5. Arrays representing $E_{\text{band}} - E_{\text{sep}}$, V_{rep} , and ΔE are filled based on the same bond distance linspace used for Task 4. They are all plotted on the same axes, and the plot is saved as a 600-DPI PNG.
6. The number of unpaired electrons is already counted by the function described in point 2, but the filling array shows the electron placements. Everywhere there's a 2, the electrons are paired, everywhere there's a 0, the orbital is unoccupied, and everywhere there's a 1, the electron is unpaired. The code searches for the ones, counts them up, and reports this count as the number of unpaired electrons in the console. Due to the presence of unpaired electrons, O_2 is paramagnetic, so if the unpaired electron counter is greater than zero, the code also prints to the console the prediction that the molecule is paramagnetic. If the number of unpaired electrons is zero, then the code would print to the console that O_2 is diamagnetic.
7. The model shows predicts the bond order of O_2 is 2, a double bond which is known to be a feature of O_2 . It also predicts paramagnetism, which is also a known feature of O_2 . The known bond length of O_2 is 1.21 angstrom, which is d_0 in the problem. It is not the consistent with the equilibrium bond distance calculated in this attempt at the model. The model equilibrium bond distance is 0.831066 angstrom. According to page 50 of *Introduction to Solid State Physics* by Charles Kittel, the cohesive energy per atom of oxygen is 2.60 eV/atom. For the two atoms in O_2 , that's 5.20 eV. This attempt at the model predicts the cohesive energy of O_2 at equilibrium bond distance to be about 25.602896 eV. This is about 5 times the actual cohesive energy. A limitation of the calibrated pair repulsion is that doesn't reproduce the equilibrium bond distance. A limitation of the tight-binding model is that it assumes the local orbitals are orthogonal (in contrast to Problem 1) and the inner-shell electrons don't play a variable roll in calculating the energies, the vectors, or the equilibrium bond distance.